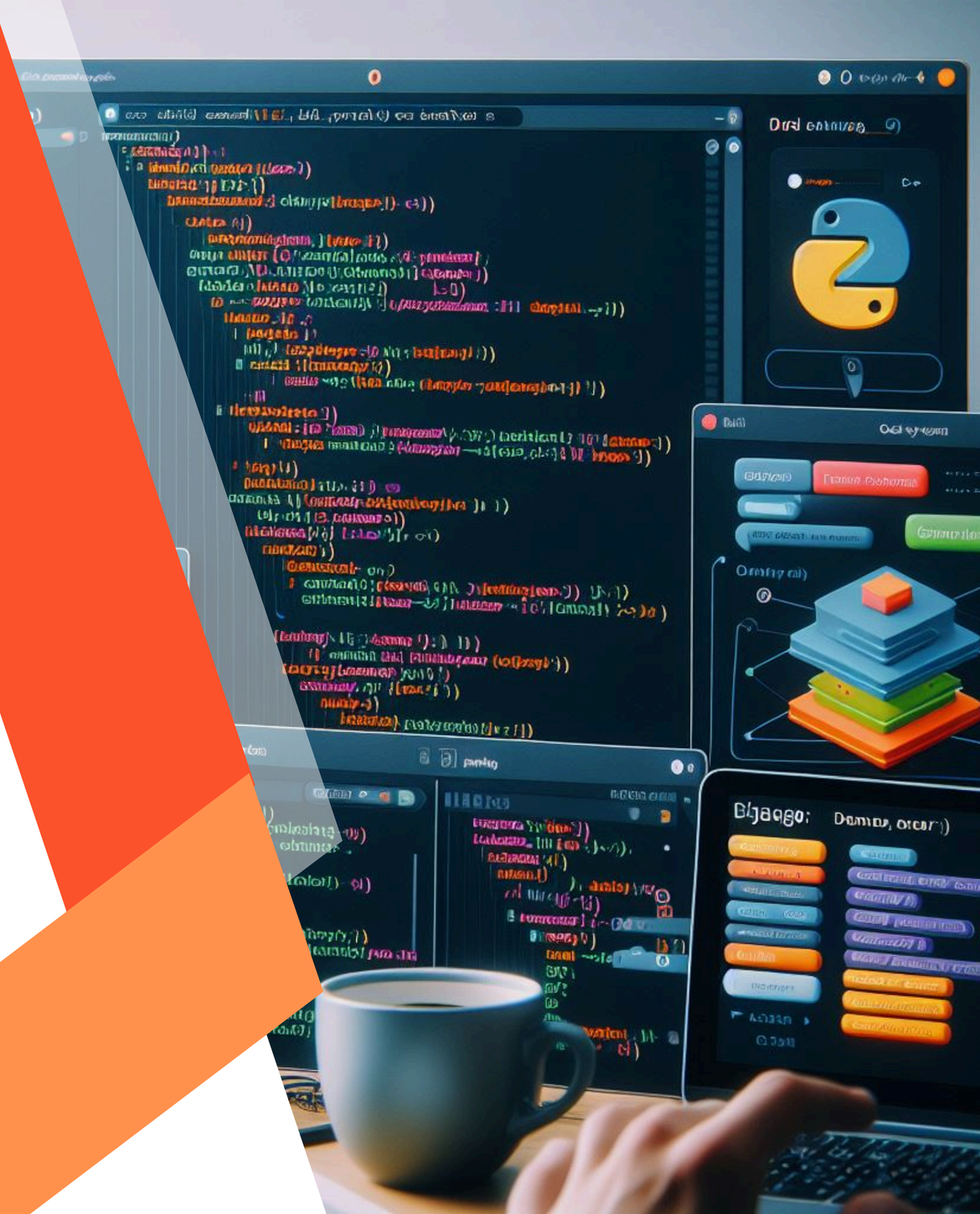
unipac.br
Barbacena

Computação Gráfica

python™

Desenhando Formas com preenchimento



PASSANDO DADOS ENTRE SHADERS

- Nesta seção, vamos criar uma aplicação que renderiza seis pontos (novamente, organizados em um padrão hexagonal) em seis cores diferentes.
- Isso requer que os dados (em particular, os dados de cor) sejam enviados do **vertex shader** para o **fragment shader**.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Os ***type qualifiers*** são palavras-chave que modificam as propriedades de uma variável, especialmente:
  - No *vertex shader,* a palavra-chave ***in*** indica que os valores serão fornecidos por um buffer, enquanto ***out*** indica que os valores serão passados para o *fragment shader*.
  - No *fragment shader*, a palavra-chave in indica que os valores serão fornecidos pelo *vertex shader*, após serem interpolados na fase de rasterização.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- A aplicação descrita usará os shaders a seguir.
- Vertex Shader:

```
1  in vec3 position;
2  in vec3 vertexColor;
3  out vec3 color;
4  void main()
5  {
6      gl_Position = vec4(position.x, position.y, position.z, 1.0);
7      color = vertexColor;
8  }
```

  - Observe a presença de duas variáveis **in**; isso ocorre porque há dois atributos de vértice, ***position*** e ***vertexColor***.
  - Os *arrays* de dados para cada um desses atributos serão armazenados em buffers separados e associados com a variável correspondente.
  - Os *arrays* devem conter o mesmo número de elementos; o tamanho dos arrays é o número de vértices.
  - Para um dado vértice, o mesmo índice do array será usado para recuperar dados de cada um dos arrays de atributos.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Fragment Shader:

```
1  in vec3 color;
2  out vec4 fragColor;
3  void main()
4  {
5      fragColor = vec4(color.r, color.g, color.b, 1.0);
6  }
```

- Aqui, a variável **in** deve ter o mesmo nome da variável **out** correspondente do vertex shader.
- Os componentes do vetor são acessados usando o sistema de nomeação (r, g, b) para facilitar a leitura, pois correspondem aos dados de cor nesse contexto.

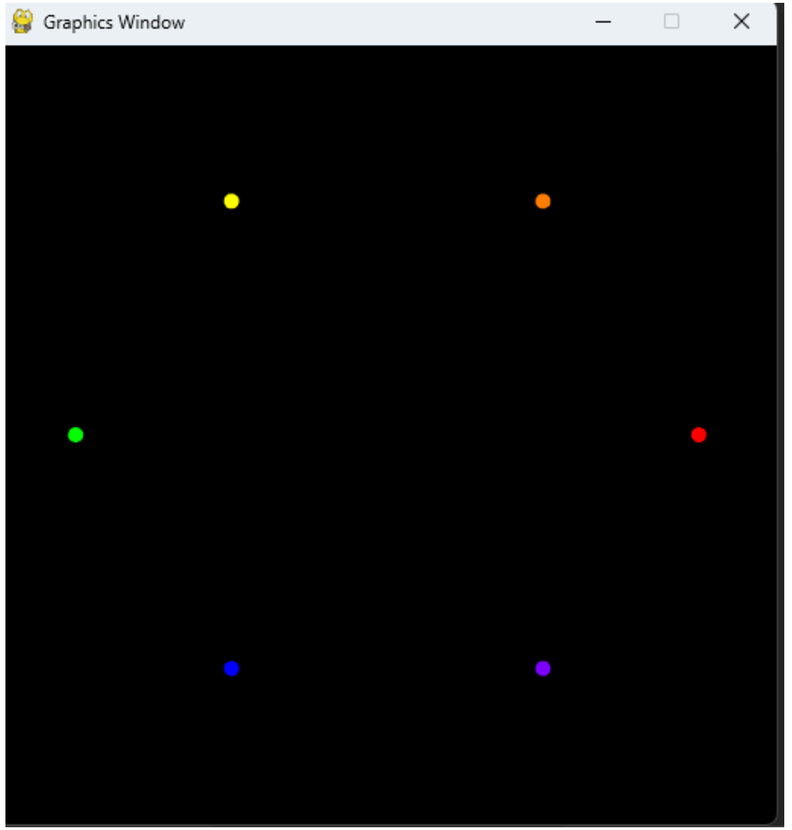
APLICAÇÃO:

test-2-4.py | test-2-2.py | test-2-1.py | test-2-3.py | GLFW.py

```python
from core.base import Base
from core.openGLUtils import OpenGLUtils
from core.Attribute import Attribute
from OpenGL.GL import *
import numpy as np  # Importar numpy para manipulação de arrays

1 usage
class Test(Base):
    def initialize(self):
        print("Initializing program...")

        vsCode = """
        in vec3 position;
        in vec3 vertexColor;
        out vec3 color;
        void main()
        {
            gl_Position = vec4(position.x, position.y, position.z, 1.0);
            color = vertexColor;
        }
        """
        fsCode = """
        in vec3 color;
        out vec4 fragColor;
        void main()
        {
```



RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Importação das Bibliotecas

```
1    from core.base import Base
2    from core.openGLUtils import OpenGLUtils
3    from core.Attribute import Attribute
4    from OpenGL.GL import *
5    import numpy as np  # Importar numpy para manipulação de arrays
6
```

- **from core.base import Base**: Importa a classe base que provavelmente contém a lógica de inicialização do OpenGL e o loop de renderização.
- **from core.openGLUtils import OpenGLUtils**: Importa funções auxiliares para a inicialização de shaders e gerenciamento de OpenGL.
- **from core.Attribute import Attribute:** Importa a classe Attribute, usada para associar os dados de vértices aos shaders.
- **from OpenGL.GL import ***: Importa todas as funções necessárias do OpenGL.
- **import numpy as np**: Importa a biblioteca numpy para manipulação eficiente de arrays de dados.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Inicialização do Programa

```
class Test(Base):
    def initialize(self):
        print("Initializing program...")

```

- **class Test(Base)::** A classe Test herda da classe Base, que fornece estrutura básica como o loop de eventos e inicialização.

- **def initialize(self)::** Método para inicializar os componentes OpenGL necessários para o programa, como *shaders* e *buffers*.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Shaders Vertex e Fragment

```
vsCode = """
in vec3 position;
in vec3 vertexColor;
out vec3 color;
void main()
{
    gl_Position = vec4(position.x, position.y, position.z, 1.0);
    color = vertexColor;
}
"""
```

```
fsCode = """
in vec3 color;
out vec4 fragColor;
void main()
{
    fragColor = vec4(color.r, color.g, color.b, 1.0);
}
"""
```

- **Vertex Shader (vsCode)**: Recebe os dados de posição e cor dos vértices. gl_Position é usado para definir a posição do vértice na tela.
- **Fragment Shader (fsCode)**: Recebe a cor interpolada entre os vértices e define a cor final dos pixels (fragmentos) renderizados.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Inicialização do Programa OpenGL

```
self.programRef = OpenGLUtils.initializeProgram(vsCode, fsCode)
```

- **OpenGLUtils.initializeProgram(vsCode, fsCode):** Função que compila e linka os shaders, retornando um identificador para o programa de shaders (programRef).

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Inicialização do Programa OpenGL

```
self.programRef = OpenGLUtils.initializeProgram(vsCode, fsCode)
```

- **OpenGLUtils.initializeProgram(vsCode, fsCode):** Função que compila e linka os shaders, retornando um identificador para o programa de shaders (programRef).

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Configurações de Renderização

```
### Configurações de renderização (opcional) ###
glPointSize(10)
glLineWidth(4)
```

- **glPointSize(10):** Define o tamanho dos pontos renderizados.
- **glLineWidth(4):** Define a espessura das linhas renderizadas.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Configuração do Vertex Array Object (VAO)

```
### Configuração do Vertex Array Object ###
vaoRef = glGenVertexArrays(1)
glBindVertexArray(vaoRef)
```

- **glGenVertexArrays(1):** Gera um identificador para o VAO.
- **glBindVertexArray(vaoRef):** Vincula o VAO para que qualquer configuração de buffer subsequente seja associada a ele.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Dados de Vértices e Atributos

```
### Dados dos Vértices e Atributos ###
positionData = np.array( object: [
    [0.8, 0.0, 0.0], [0.4, 0.6, 0.0], [-0.4, 0.6, 0.0],
    [-0.8, 0.0, 0.0], [-0.4, -0.6, 0.0], [0.4, -0.6, 0.0]
], dtype=np.float32)

colorData = np.array( object: [
    [1.0, 0.0, 0.0], [1.0, 0.5, 0.0], [1.0, 1.0, 0.0],
    [0.0, 1.0, 0.0], [0.0, 0.0, 1.0], [0.5, 0.0, 1.0]
], dtype=np.float32)
```

- **positionData:** Contém a posição dos vértices em 3D.
- **colorData:** Contém as cores correspondentes a cada vértice (RGB).

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Atributos dos Vértices

```
# Atributo de posição
positionAttribute = Attribute( dataType: "vec3", positionData)
positionAttribute.associateVariable(self.programRef,  variableName: "position")

# Atributo de cor
colorAttribute = Attribute( dataType: "vec3", colorData)
colorAttribute.associateVariable(self.programRef,  variableName: "vertexColor")
```

- **Attribute("vec3", positionData):** Cria um objeto de atributo para a posição dos vértices e outro para a cor.
- **associateVariable(self.programRef, "position"):** Associa o atributo da posição ao shader vertex.
- **associateVariable(self.programRef, "vertexColor"):** Associa o atributo da cor ao shader vertex.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Buffer de Vértices (VBO)

```
# Gerar e associar o buffer de vértices (VBO)
self.vertexCount = len(positionData)
vbo = glGenBuffers(1)
glBindBuffer(GL_ARRAY_BUFFER, vbo)
# Concatenar os dados de posição e cor em um único array
vertexData = np.hstack([positionData, colorData])  # Concatenando os arrays
glBufferData(GL_ARRAY_BUFFER, vertexData.nbytes, vertexData, GL_STATIC_DRAW)
```

- **lGenBuffers(1):** Gera um identificador para o buffer de vértices.
- **glBindBuffer(GL_ARRAY_BUFFER, vbo):** Associa o buffer à OpenGL para que os dados possam ser carregados.
- **np.hstack([positionData, colorData]):** Junta os arrays de posição e cor em um único array, para enviar ambos os dados em um só buffer.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Associando o VBO ao VAO

```
# Associar o buffer ao VAO
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * positionData.itemsize, None)
glEnableVertexAttribArray(0)

glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * positionData.itemsize, ctypes.c_void_p(3 * positionData.itemsize))
glEnableVertexAttribArray(1)

glBindBuffer(GL_ARRAY_BUFFER,  buffer: 0)
glBindVertexArray(0)
```

- **glVertexAttribPointer:** Define a estrutura dos dados no VBO (os dados de posição e cor).
- **glEnableVertexAttribArray:** Habilita o uso dos atributos de vértices no shader.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Desenhando os Vértices

```
# Usar o programa de shaders
glUseProgram(self.programRef)
glBindVertexArray(1)  # Certifique-se de que o VAO correto está vinculado
glDrawArrays(GL_POINTS, first: 0, self.vertexCount)
glBindVertexArray(0)
```

- **glUseProgram(self.programRef):** Usa o programa de shaders ativo.
- **glBindVertexArray(1):** Associa o VAO.
- **glDrawArrays(GL_POINTS, 0, self.vertexCount):** Desenha os pontos usando os dados dos vértices.
- **glBindVertexArray(0):** Desvincula o VAO.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Loop de Entrada e Encerramento. Execução do Programa.

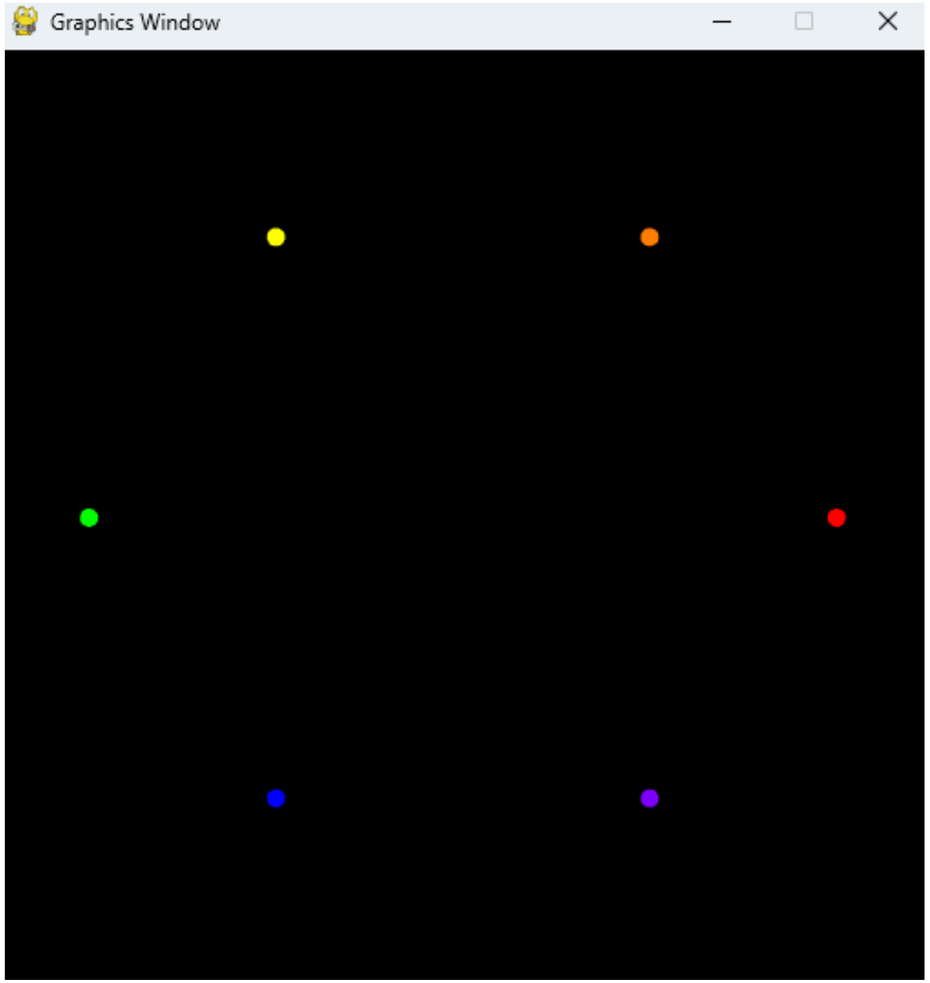
```
        self.input.update()
        # Verifica se o usuário fechou a aplicação
        if self.input.quit:
            self.running = False

# Instanciando a classe e rodando o programa
Test().run()
```

- **self.input.update():** Atualiza o estado de entrada (ex: teclado, mouse).
- **if self.input.quit::** Verifica se o usuário solicitou o fechamento da aplicação.
- **Test().run()**: Cria uma instância da classe Test e executa o loop principal do programa.

RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- O resultado da execução deste aplicativo é ilustrado na Figura



RELEMBRANDO SOBRE A OPENGL SHADING LANGUAGE:

- Lembre-se do aula anterior que durante o pipeline gráfico, entre o estágio de processamento de geometria (que envolve o shader de vértice) e o estágio de processamento de pixel (que envolve o shader de fragmento) está o estágio de rasterização.
- Neste ponto, os programas são suficientemente complexos para ilustrar a interpolação de atributos de vértice que ocorre durante este estágio.
- Durante o estágio de rasterização, os pontos são agrupados em primitivos geométricos: pontos, linhas ou triângulos, dependendo do modo de desenho especificado.

INTERPOLAÇÃO DE ATRIBUTOS DE VÉRTICES

- **Interpolação**: Durante a rasterização, os atributos dos vértices (como cor) são interpolados.
- Exemplo: Um ponto entre dois vértices com cores diferentes será colorido com uma média ponderada dessas cores, de acordo com sua distância.

INTERPOLAÇÃO DE ATRIBUTOS DE VÉRTICES

- **Exemplo de Interpolação de Cores**
- **Segmento de linha P1 e P2:**
  - P1: Cor [1, 0, 0] (vermelho)
  - P2: Cor [0, 0, 1] (azul)
  - O ponto no meio será interpolado para [0.5, 0, 0.5], uma cor roxa.
  - Outro ponto a 25% da distância terá cor [0.75, 0, 0.25], um roxo avermelhado.

INTERPOLAÇÃO DE ATRIBUTOS DE VÉRTICES

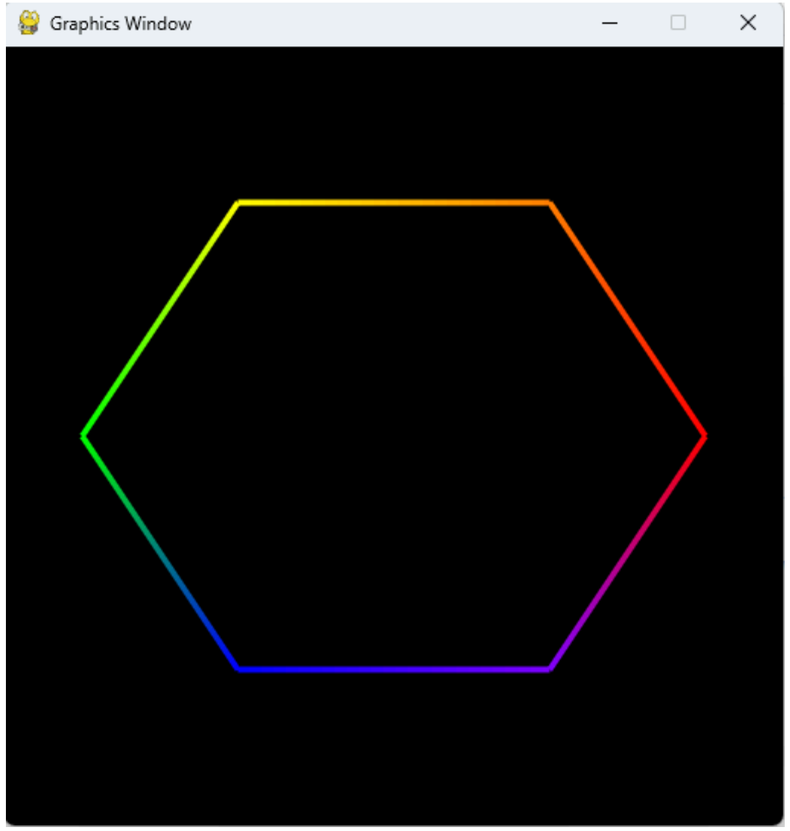
- Para implementar a interpolação dos pontos no programa, será necessário alterar o modo de desenho de GL_POINTS para outro modo que suporte a interpolação, como **GL_LINE_LOOP** ou **GL_TRIANGLE_FAN**, além de garantir que o OpenGL interpole automaticamente as cores entre os vértices.

INTERPOLAÇÃO DE ATRIBUTOS DE VÉRTICES

- Alterar o modo de desenho para interpolação:
- No método update, em vez de usar GL_POINTS, altere o modo para GL_LINE_LOOP ou GL_TRIANGLE_FAN, dependendo do tipo de geometria que você deseja desenhar:

```
# Desenhar os vértices com interpolação de cores usando GL_LINE_LOOP
glDrawArrays(GL_LINE_LOOP, first: 0, self.vertexCount)
```

INTERPOLAÇÃO DE ATRIBUTOS DE VÉRTICES



INTERPOLAÇÃO DE ATRIBUTOS DE VÉRTICES

```
# Desenhar os vértices com interpolação de cores usando GL_LINE_LOOP
glDrawArrays(GL_TRIANGLE_FAN,  first: 0, self.vertexCount)
```

